

SentinelSleep — Real-Time Audio AI for PTSD Nightmare Intervention

Final Report · MS Data Science · UMKC · Spring 2026

Nithin Reddy · reddy.nithin.0331@gmail.com

May 2026

Live App: sentinal-sleep.vercel.app · **Demo Video:** [Google Drive](#) · **Slide Deck:** [Google Slides](#)
· **Repository:** github.com/reddy-nithin/SentinalSleep · **Decision log:** EVIDENCE_LOG.md
· **Architecture:** docs/architecture.md

Executive Summary

SentinelSleep is an edge-deployed audio AI system that detects PTSD nightmare signatures from bedroom microphone audio in real time and injects pre-generated therapeutic soundscapes at low volume (-20 dBFS) to interrupt nightmares without waking the patient. It operationalizes Targeted Memory Reactivation (TMR) research: low-volume auditory cues during REM sleep can influence sleep stage transitions without causing arousal.

The system addresses a concrete coverage gap — 8 million Americans live with chronic PTSD nightmares, and existing interventions require pharmacological tolerance, trained clinician access, or disruptive wake-based hardware. SentinelSleep runs entirely on a MacBook M2 (Apple Silicon), costs nothing to operate beyond hardware, and generates objective overnight clinical data where today only subjective patient recall exists.

The prototype is built across seven complete phases: audio detection (MIT AST), emotional verification (audeering wav2vec2), generative audio caching (MusicGen + AudioGen), real-time orchestration (pure state machine + SQLite), a morning review dashboard (Next.js 16 + Vercel), a simulation runner for demos, and a documentation and portfolio polish layer. Key measured outcomes: 620 ms inference latency per 2-second chunk on M2 MPS, ~150 passing tests across 12 pytest files, and an end-to-end demo state arc of 31 seconds. **This is a research prototype, not a medical device.** Clinical efficacy has not been measured and would require an IRB-approved sleep study.

Technical Approach

Pipeline overview. The system processes audio in a 2-second rolling buffer and passes each chunk through a four-layer cascade. Layer 1 (Detection) feeds the chunk into the MIT Audio Spectrogram Transformer (AST), fine-tuned on AudioSet-527, computing a Distress Signal Score (DSS) across eight distress-relevant classes (crying, whimpering, screaming, breathing-heavy, gasp, groan, rustle, thump). Layer 2 (Verification) activates only when $DSS > 0.4$ AND ≥ 2 distress classes co-occur; it runs audeering wav2vec2 to extract continuous valence, arousal, and dominance and confirms a nightmare signature when valence < 0.4 , arousal > 0.6 , and dominance < 0.4 hold for ≥ 15 sustained seconds. Two-stage verification is what gives the system false-positive resistance that a single large model would not provide. Layer 3 (Intervention) plays a pre-cached WAV selected from a 16-clip library (5 mild $\times$ 5 severe variants, mixed with pydub at -20 dBFS). Layer 4 (Escalation) triggers a progressive wake protocol if distress persists beyond 60 seconds.

Model selection. MIT AST was chosen for AudioSet label coverage and MPS compatibility. Audeering wav2vec2 was selected because dimensional emotion regression (continuous V/A/D) provides richer verification signal than categorical classification. MusicGen-small and AudioGen-medium generate the music and soundscape cache layers; AudioGen replaced AudioLDM2 mid-build when AudioLDM2 was deprecated upstream (ADR-014). All four models are pre-trained and used without fine-tuning in this prototype.

DSS formulation. $DSS = \sum w_i \cdot p_i$ over the eight distress classes, where p_i is the AST-predicted probability and w_i is a hand-tuned weight. A multi-class co-occurrence guard (≥ 2 active classes per chunk) prevents single-source false positives from speech audio, TV audio, or ambient noise.

Memory budget and MPS targeting. The live detection loop holds only AST (~350 MB) and wav2vec2 (~660 MB) — approximately 1 GB on M2 8GB unified memory. MusicGen and AudioGen are too large for the live budget and run exclusively in `scripts/pregenerate_cache.py`, which is designed to run on Colab T4 and then exit. This is not a compromise — offline generation produces cleaner separation of concerns and makes the live loop dramatically more testable. All config constants (`DSS_FLAG_THRESHOLD = 0.4`, `NIGHTMARE_CONFIRM_DURATION_SECONDS = 15`, `INTERVENTION_PLAYBACK_DBFS = -20.0`, etc.) live exclusively in `config.py`. No magic numbers elsewhere.

State machine and SQLite logging. The orchestrator implements a pure state machine: LISTENING $\rightarrow$ FLAGGED $\rightarrow$ INTERVENING $\rightarrow$ ESCALATING $\rightarrow$ RESOLVED. Every state transition writes to SQLite *before* any audio side effect — this is a safety invariant enforced in code, not by convention. The event logger uses parameterized queries only; no string interpolation in SQL.

Dashboard isolation. The Next.js 16 morning dashboard reads from JSON snapshots exported by `scripts/export_snapshot.py`. It never touches `events.db` directly. `queries.py` contains only SELECT statements. This enforces a strict read-only contract between the UI and the event log.

Experiments

Detection separation. DSS scores across fixture categories show clear separation: calm sleep audio produces DSS 0.01–0.08; single non-distress events (isolated rustle, single breath) score 0.05–

0.15; multi-class nightmare fixtures (co-occurring whimper + heavy breathing + groan) score 0.45–0.78 — well above the 0.40 flag threshold. The multi-class co-occurrence rule was the key design decision that achieved this separation; a single-class threshold would have produced substantially more false positives on ambient audio.

Latency benchmark. Per-chunk inference on M2 MPS averages 620 ms at the 50th percentile against a 700 ms budget (2-second chunk allows up to 700 ms processing before the next chunk would be late). The 80ms headroom handles occasional memory pressure without dropping chunks.

End-to-end simulation. The demonstration session (session_id=4, seeded in `web/data/sessions.json`) traces a full state arc in 31 seconds: LISTENING at t=0 (DSS 0.05), FLAGGED at t=3 (DSS spike), INTERVENING at t=18 (15-second sustained nightmare confirmed), RESOLVED at t=31 (distress markers normalized after clip playback begins). This matches the designed 15-second confirmation window.

Test suite. Approximately 150 tests across 12 pytest files cover the full pipeline: detection (AST classifier, DSS formula, multi-class thresholds), verification (EmotionModel, continuous-streak semantics, confidence calculation), mixer (pydub layer mixing, volume calibration), state machine (state transition coverage, logging order invariant), event logger (SQLite schema, parameterized queries), clip selector (confidence-based selection from cache), and integration runner (stream → detection → verification → playback). Run with `uv run pytest tests/ --cov=sentinel.sleep`.

Honest limitations. All evaluation was performed on synthetic fixtures and AudioSet-derived recordings. Real PTSD recordings are scarce, ethically gated, and require IRB approval to collect. Co-sleeping scenarios, room reverberation effects, and snoring vs. distress classification have not been characterized. Clinical efficacy — whether the 60-second therapeutic audio actually reduces nightmare frequency or severity — requires a controlled sleep study. The escalation layer uses fixed thresholds rather than adaptive per-patient calibration.

Lessons Learned

Memory budgets shape architecture more than accuracy. The decision to run generation offline (ADR-003, documented Week 3) was initially driven by hardware necessity — MusicGen and AudioGen together exceed the M2 8GB live budget. What emerged from that constraint was cleaner than what free-choice architecture would have produced: a cache layer with provenance tracking, a manifest with SHA-256 content hashes, and a live loop that has no dependency on model loading at runtime. The constraint forced a better design.

Two-stage verification beats one bigger model. The original architecture sketch considered a single end-to-end nightmare classifier. Using AST for broad distress detection and wav2vec2 for semantic emotional confirmation — with independent thresholds and the 15-second sustained-streak rule — produced qualitatively stronger false-positive resistance. Each layer is also independently testable and independently replaceable. The cascade design was validated by ADR-005; it was the right call.

Decision logs scale better than memory. The 18-entry EVIDENCE_LOG.md (ADR-001 through ADR-018) accumulated organically across the project. When AudioLDM2 was deprecated mid-build (ADR-014), the pivot to AudioGen took under an hour — because the original rationale for

AudioLDM2 was documented and showed it was entirely substitutable. Without the log, a three-week gap between sessions would have cost significant re-investigation time. For any multi-week ML project built under intermittent focus, an ADR-style decision log is as important as the test suite.

Career Relevance

SentinelSleep sits at the exact intersection I am targeting for full-time roles after graduating in May 2026: applied healthcare AI, generative multimodal models on edge hardware, and end-to-end ML system delivery from research ingestion through to a deployed dashboard.

Healthcare AI fluency. This project complements my role as co-founder of TruPharma Clinical Intelligence, which won 1st place in the AI/DS Track at UMKC Research-a-thon 2026. Both projects share the discipline of operating responsibly in clinical contexts — building toward patient benefit without overclaiming, maintaining honest evaluation standards, and treating the gap between “engineering metric” and “clinical evidence” as a first-class concern rather than a footnote. Healthcare AI built without that discipline is actively harmful; building it with that discipline is the skill I am developing.

Generative AI engineering. SentinelSleep gave me concrete, production-oriented experience with MusicGen and AudioGen — not as a demo, but as a cache generation pipeline with provenance tracking, fallback strategies (synthetic pink-noise for CI stubs), and SHA-256 manifest integrity verification. Running large generative models on resource-constrained edge hardware requires the same cost-awareness and architecture discipline as running them on cloud APIs. The offline cache pattern (ADR-003) is a reusable solution to this class of problem across many healthcare AI contexts where real-time model inference is too expensive, too slow, or too unpredictable.

End-to-end delivery. I owned the full stack: research literature to architecture, four model integrations, SQLite schema design, state machine, Next.js dashboard, Vercel deployment, and documented decision-making throughout. This is the profile I am bringing to data science and healthcare AI roles — including the KCMO civic data science internship for which I have applied. The ability to take an ambiguous clinical problem from concept to reproducible prototype, with honest documentation of what was and was not validated, is exactly what that kind of applied role requires.

Most importantly, building SentinelSleep taught me that responsible healthcare AI is mostly about what you decide not to ship — and how clearly you document that line.

Repository: github.com/reddy-nithin/SentinalSleep

Live App: sentinal-sleep.vercel.app

Demo Video: [Google Drive](#)

Slide Deck: [Google Slides](#)

Decision log: EVIDENCE_LOG.md · **Architecture:** docs/architecture.md

Models: MIT AST · audeering wav2vec2 · MusicGen · AudioGen

License attribution: NOTICES.md
